NoSQL Database — BCSE406L · Review 2

Real-Time Collaborative Whiteboard

An append-only document model in MongoDB, with Redis for ordering, presence and cross-instance event fanout

80,019
stroke documents in the seeded database

7
purpose-built indexes, every plan measured

9
aggregation pipelines, live in the application

3
collections: rooms, strokes, snapshots

Devesh Atul Mahajan (23BCE0801) · Prakrit Bahl (23BCE0826) · Lakshya Diwani (23BCE0811)

Batch NS02 · B.Tech CSE · Guide: Dr. D. Vivek

AGENDA

What this deck argues, and where

Rubric criterion	Marks	Argued by	Slides
System design	6	Append-only model, two-store split, ordering, fanout	3 – 6
Database implementation	8	Collections, flexible schema, CRUD, deletion, indexes	7 – 12
Query quality	6	Measured index plans, nine pipelines, join cost	13 – 17
Prototype / demo	5	Running system, cross-instance sync, live analytics	18 – 20
Documentation & progress	5	Deviations, limitations, plan for Review 3	21 – 22

Everything shown is measured against a running system.

The numbers in this deck come from docs/measurements.json, produced by a script that queries the live application. No figure here was typed by hand.

SYSTEM DESIGN

One decision shapes everything else

The canvas is never stored as a picture

It is an ordered, append-only stream of stroke documents. What a user sees is derived state, rebuilt by replaying that stream.

Why that makes it a NoSQL project

Four tools produce genuinely different documents in one collection. Undo becomes a flag, so history survives. Ordering cannot come from a clock.

Replay, not snapshots of truth. A snapshot is only a cache; the log remains the system of record.

Undo is a soft delete. active: false retains the stroke, so redo and analytics still see it.

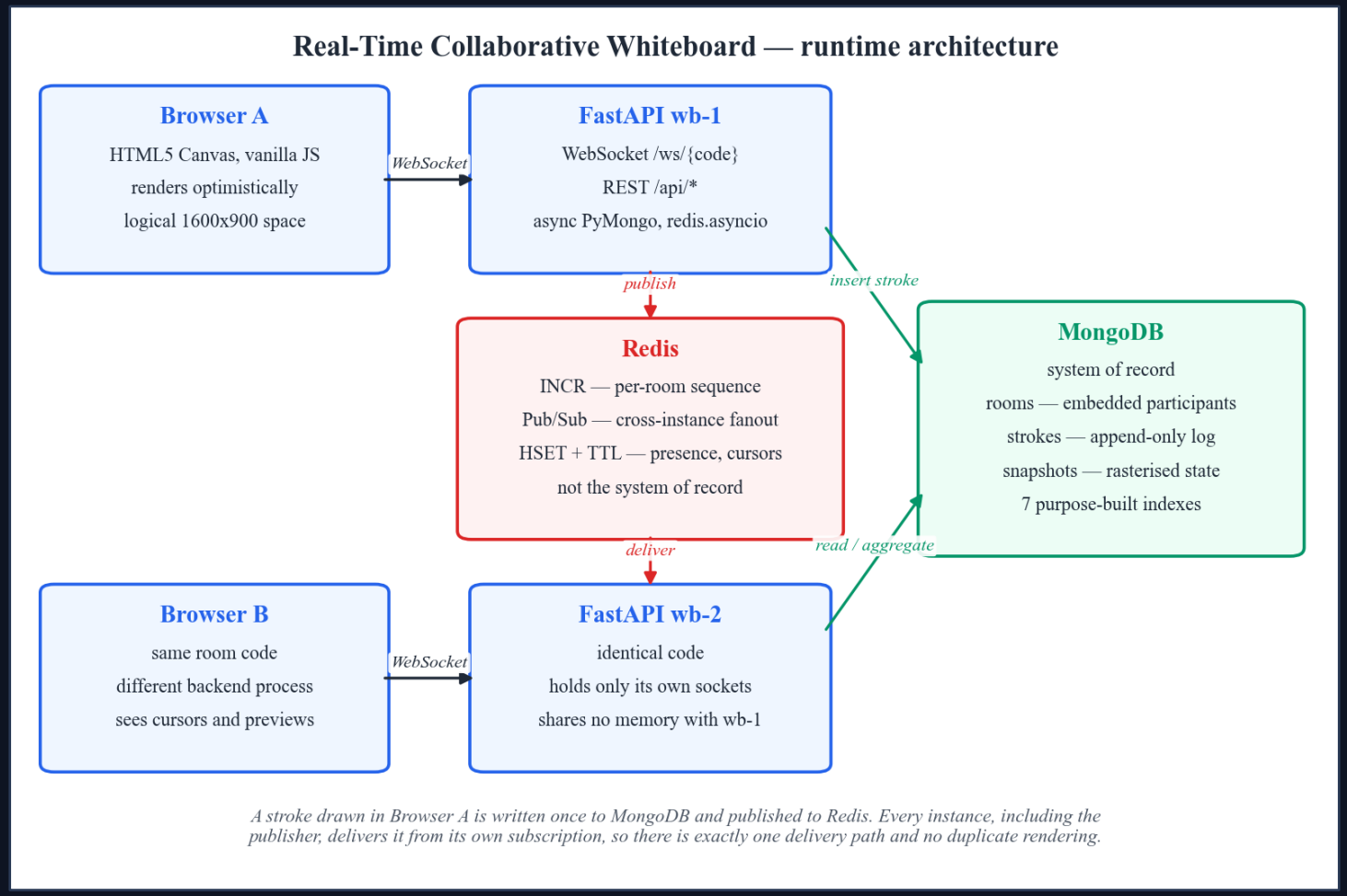
Order comes from Redis INCR. Atomic across every instance, so replay is total and reproducible.

Previews are never persisted. One stroke writes exactly one document, however many frames it took to draw.

Sections 1 and 2 of the report

SYSTEM DESIGN

Runtime architecture



Two FastAPI processes, sharing no memory. This is deliberate.

A single process could broadcast from a dictionary and would never exercise the fanout layer at all.

MongoDB is the only durable state: rooms, the stroke log, snapshots.

Redis is the bus and the counter — drawn between the instances because that is its role.

Both drivers are async. A blocking call in a WebSocket handler would stall every other user on that process.

Demonstrated live: two browsers, two processes, one board

SYSTEM DESIGN

Why two data stores, and what breaks

	MongoDB	Redis
Holds	Rooms, stroke log, snapshots	Counters, Pub/Sub, presence
Durability	System of record	Disposable cache
If it restarts	Boards are gone — this is the data that matters	Cursors vanish, presence rebuilds, counter realigns from Mongo. No drawing lost.
Why not the other one	Cannot give an atomic cross-process counter cheaply	Cannot be trusted with the only copy of a board

The division is by durability requirement, not convenience.

A cursor position that is stale within 16 ms is write amplification for no benefit, so it expires by TTL in Redis and is never written to disk.

SYSTEM DESIGN

Ordering: the subtlest part of the design

Candidate	Verdict	Why
Client timestamp	Rejected	Participants' clocks disagree, sometimes by seconds
ObjectId timestamp	Rejected	One-second granularity: two strokes 40 ms apart order arbitrarily
Redis INCR per room	Chosen	Atomic across every instance, so the order is total and reproducible

app/repositories/strokes.py

```
seq = await r().incr(f"room:{room_id}:seq")
# Redis is a cache, not the record, so a restart must not
# collide: realign upward from the highest persisted seq.
highest = await db()[STROKES].find_one(
    {"roomId": room_id}, sort=[("seq", -1)], projection={"seq": 1})
```

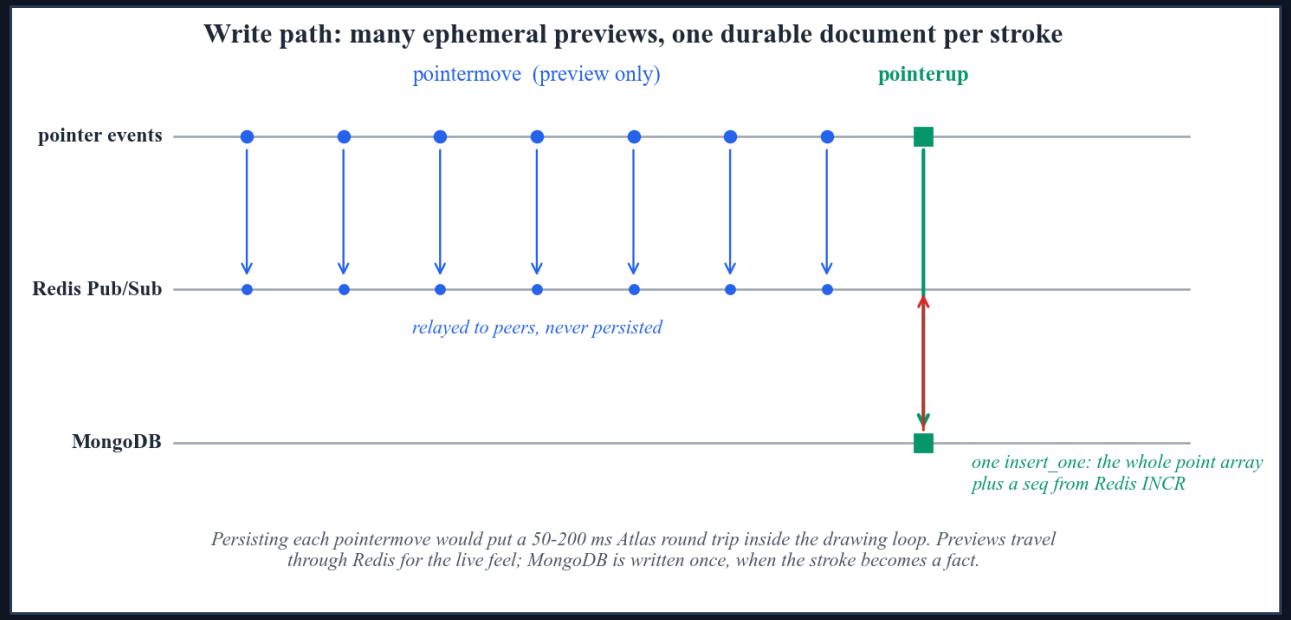
Why it matters: replay only rebuilds the right picture in the right order.

Failure mode avoided: an intermittent mis-ordering that is very easy to miss in testing.

Second reason for Redis. Ordering, not just fanout, is why it is in the stack.

SYSTEM DESIGN

The write path, and the duplicate-stroke trap



Exactly one delivery path

The obvious version is wrong. Send to local clients and also publish to Redis, and the publisher — which is subscribed — renders the stroke twice.

Invisible on an opaque pen. Obvious with a transparent stroke or an eraser, and easily misread as a rendering bug.

So handlers never write to their own sockets. Every instance, publisher included, delivers from its subscription.

The author is excluded by an id carried inside the published message, so it works whichever instance holds that socket.

Section 2.7 of the report

DATABASE IMPLEMENTATION

Three collections, measured

Collection	Holds	Documents	Data	Indexes
rooms	One document per board, participants embedded	8	0.00 MB	0.11 MB
strokes	The append-only log, one document per stroke	80,019	120.3 MB	9.8 MB
snapshots	Rasterised canvas at a known sequence number	5	13.8 MB	0.04 MB

1,576 B

average stroke document — the basis of the capacity estimate

8

rooms, one of them deliberately large for the measurements

2 → 3

collections: snapshots was added after measuring join cost

Review 1 proposed two collections. Measurement added the third.

A full replay of the largest room transfers 66 MB, so snapshotting moved from 'optional' to part of the normal join path.

DATABASE IMPLEMENTATION

Flexible schema, shown rather than claimed

Three documents from the same collection, written by the same code path. No migration, no nullable columns standing in for absent fields.

pen — carries points

```
{
  "tool": "pen",
  "seq": 1,
  "points": [
    {"x": 323.7, "y": 294.2},
    {"x": 325.9, "y": 293.9},
    ... 145 more ...
  ],
  "pointCount": 148,
  "style": {"width": 6.4},
  "active": true
}
```

rect — no points at all

```
{
  "tool": "rect",
  "seq": 14,
  "geometry": {
    "x": 798.1, "y": 159.3,
    "w": 417.5, "h": -191.1
  },
  "bbox": {"minX": 798.1, ...},
  "style": {"width": 8.3},
  "active": true
}
```

text — text and font

```
{
  "tool": "text",
  "seq": 21,
  "text": "stroke log =",
  "font": "600 34px Inter",
  "geometry": {
    "x": 452.0, "y": 118.0
  },
  "style": {"color": "#e8ecf4"},
  "active": true
}
```

Adding a polygon tool tomorrow needs a model and a renderer — no migration. Existing documents are untouched and replay keeps working, because readers only look at the fields relevant to each tool.

Sections 3.3 and 3.7 of the report

Embed or reference, and the Redis key space

The governing question is whether the child is bounded

Relationship	Strategy	Why
Room → participants	Embedded	Bounded, small, read with the room
Room → strokes	Referenced	Unbounded — 42,672 in one room
Room → snapshots	Referenced	Large binaries, pruned separately

Redis is a database here too, so its keys are documented

Key	Type	Lifetime
room:{id}:seq	Counter	Persistent, realigned on join
wb:room:{CODE}	Pub/Sub	Transient, never stored
room:{CODE}:presence	Hash	Expires after 2 min of silence

No amount of tuning makes an unbounded embedded array safe.

At 1,576 bytes per stroke, the largest room alone would be far past the 16 MB document limit. Referencing also lets one stroke be undone or restyled without rewriting the room.

The full CRUD surface, over REST and WebSocket

An append-only log has no natural update or delete. These were designed in, not bolted on — and every one is reachable in Postman.

	Endpoints	What it does
C	POST /api/rooms · POST /api/rooms/{code}/strokes	Room with a unique code; one stroke document
R	GET /api/rooms · /rooms/{code}/strokes?afterSeq= · /strokes/{id}	List, full replay, incremental tail, single document
U	PATCH /api/rooms/{code} · PATCH /api/strokes/{id} · POST /strokes/{id}/move	Rename; restyle nested fields; $inc nested geometry
D	POST /rooms/{code}/undo · redo · compact · DELETE /strokes/{id} · DELETE /rooms/{code}	Soft delete, hard delete, and cascade

A stroke created over REST appears on an open board immediately, because the endpoint publishes to Redis — a far better demonstration than a refresh.

Verified by script, not by assertion. A run walks every operation including the concurrent-undo race and reports pass or fail.

Three deletion semantics, and the one real race

Semantics	Retains data	Used for
Soft delete	Yes, fully	Undo and redo
Compaction	No	Reclaiming finished boards
Cascade delete	No	Removing a board entirely

MongoDB has no ON DELETE CASCADE.

Deleting a room without its strokes would silently orphan tens of thousands of documents no query returns and no user can reach. Referential integrity is the application's job, and the endpoint reports what it removed.

undo_last — one atomic claim

```
# Scoped to the user: a room-wide undo would let one
# person retract another's work.
# Atomic: find-then-update in two calls lets two
# simultaneous undos claim the same stroke.
doc = await db()[STROKES].find_one_and_update(
    {"roomId": rid, "userId": uid, "active": True},
    {"$set": {"active": False, "undoneAt": now}},
    sort=[("seq", -1)],
    return_document=ReturnDocument.AFTER,
)
```

The second request no longer matches active: true on that document and moves to the next one. This is also the query the dedicated undo index exists for.

QUERY QUALITY

Seven indexes, each for a named query

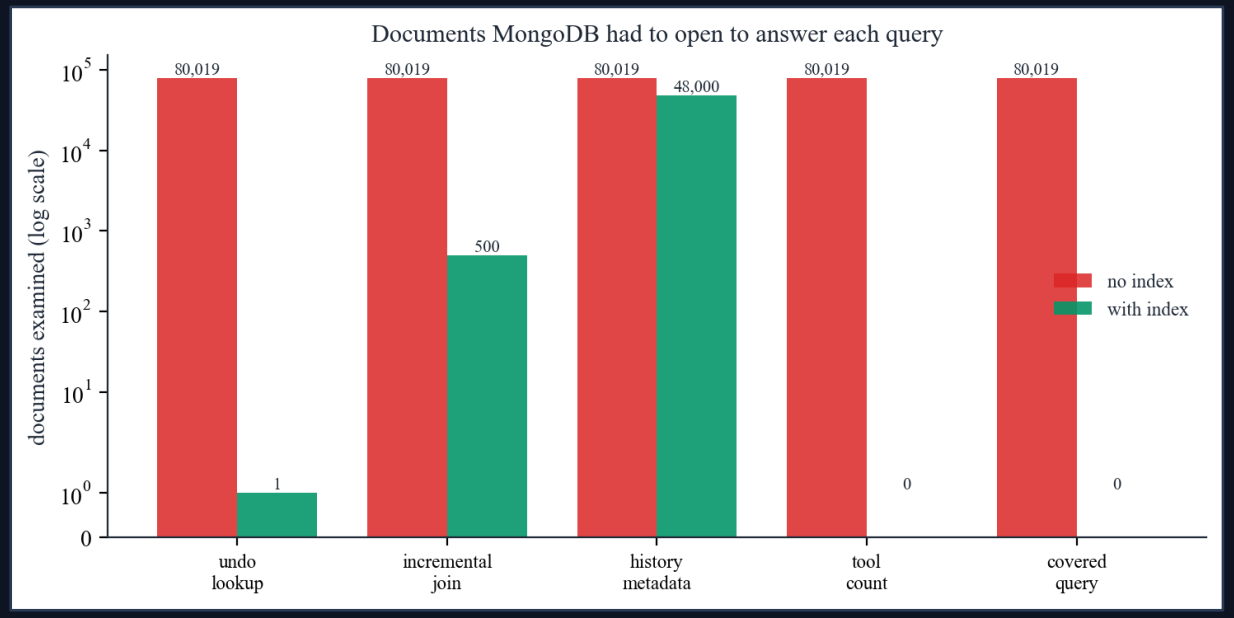
Index	Key	The query it exists for
ix_room_code	{code: 1}	Every join is a lookup by short room code; unique prevents duplicate codes.
ix_room_activity	{lastActivityAt: -1}	Powers the 'recent rooms' list without scanning the whole collection.
ix_stroke_replay	{roomId: 1, seq: 1}	The hot path: replay a room's strokes in order. Covers range queries on seq for incremen
ix_stroke_undo	{roomId: 1, userId: 1, active: 1, seq: -1}	Per-user undo claims the newest active stroke belonging to that user. Descending seq let
ix_stroke_tool	{roomId: 1, tool: 1}	Supports the tool-distribution aggregation without a collection scan.
ix_stroke_time	{roomId: 1, createdAt: 1}	Supports the time-bucketed activity aggregation ($dateTrunc grouping).
ix_snapshot_latest	{roomId: 1, upToSeq: -1}	Fetches the newest snapshot for a room in a single index seek.

None was added speculatively.

Indexes cost storage and slow every write. These total 9.8 MB against 120 MB of data (8% overhead). The largest single index is _id, which no query in this application ever uses.

QUERY QUALITY

Indexes: measured, not asserted



Documents MongoDB had to open, log scale. Planner output, so it is reproducible.

Query	Without index	With index	Gain
undo lookup	COLLSCAN · 80,019	IXSCAN · 1	23.2x
incremental join	COLLSCAN · 80,019	IXSCAN · 500	3.7x
history metadata	COLLSCAN · 80,019	IXSCAN · 48,000	1.1x
tool count	COLLSCAN · 80,019	COUNT_SCAN · 0	4.8x
covered query	COLLSCAN · 80,019	IXSCAN · 0	1.7x

The method

Drop the indexes, run each real query five times keeping the best, capture explain in executionStats mode, rebuild, repeat.

Two queries reach zero documents examined: the count wins a COUNT_SCAN that tallies index keys, and the covered query is answered entirely from the index.

Section 5.3 of the report

QUERY QUALITY

Where the index barely helps

history metadata: 305 ms without → 274 ms with (1.1x)

It still opens 48,000 documents. This result is reported rather than omitted.

The query is not selective. It returns tens of thousands of documents, so the index lets the server skip almost nothing.

Index then fetch is random access; scanning the collection in storage order is sequential. When a query touches most of a collection, the sequential scan wins.

MongoDB's own planner would often choose the scan. The optimiser agreeing with the measurement is the point.

The general rule: an index pays off in proportion to how much of the collection it lets the server skip. A query that skips nothing gains nothing.

Timings are also cache-warm: the whole collection fits in the WiredTiger cache here, which flatters the unindexed case. On Atlas M0 the gap would be wider, so these speed-ups are a conservative floor.

Section 5.4 of the report

QUERY QUALITY

Nine pipelines, each answering a real question

Pipeline	Question it answers	Stages	Time
Contribution breakdown by user	Who drew how much in this room, and how often did they und	$match, $group, $project, $sort	351 ms
Tool usage mix	Which tools dominate, and how complex is a typical stroke	$match, $group, $project, $sort	166 ms
Drawing activity over time	How did drawing intensity vary minute by minute?	$match, $group, $project, $sort, $limit	159 ms
Stroke complexity distribution	Are strokes mostly short taps or long freehand curves?	$match, $bucket, $project	238 ms
Colour palette in use	Which colours carry the most ink on the active canvas?	$match, $group, $project, $sort, $limit	188 ms
Content bounding box	What canvas region actually contains drawing?	$match, $group, $project	191 ms
Room leaderboard (joined)	Across all rooms, which are the busiest?	$match, $lookup, $set, $project, $sort, $limit	354 ms
Single-request dashboard	Can the whole analytics panel load in one query?	$match, $facet	420 ms
Storage footprint by tool	How many bytes does each kind of stroke document cost?	$match, $set, $group, $project, $sort	219 ms

Not decorative: every one is reachable from the analytics panel in the running application, which displays the pipeline next to its own output.

QUERY QUALITY

Two pipelines worth arguing about

$lookup with a sub-pipeline

```
{"$lookup": {
   "from": "strokes",
   "localField": "_id",
   "foreignField": "roomId",
   "as": "agg",
   "pipeline": [
     {"$match": {"active": true}},
     {"$group": {"_id": null,
        "total": {"$sum": 1},
        "people": {"$addToSet": "$userId"}}}
   ]}}
```

The sub-pipeline aggregates inside the join, so tens of thousands of strokes are never materialised into the room document. Without it, the array would exceed 16 MB and the query would fail.

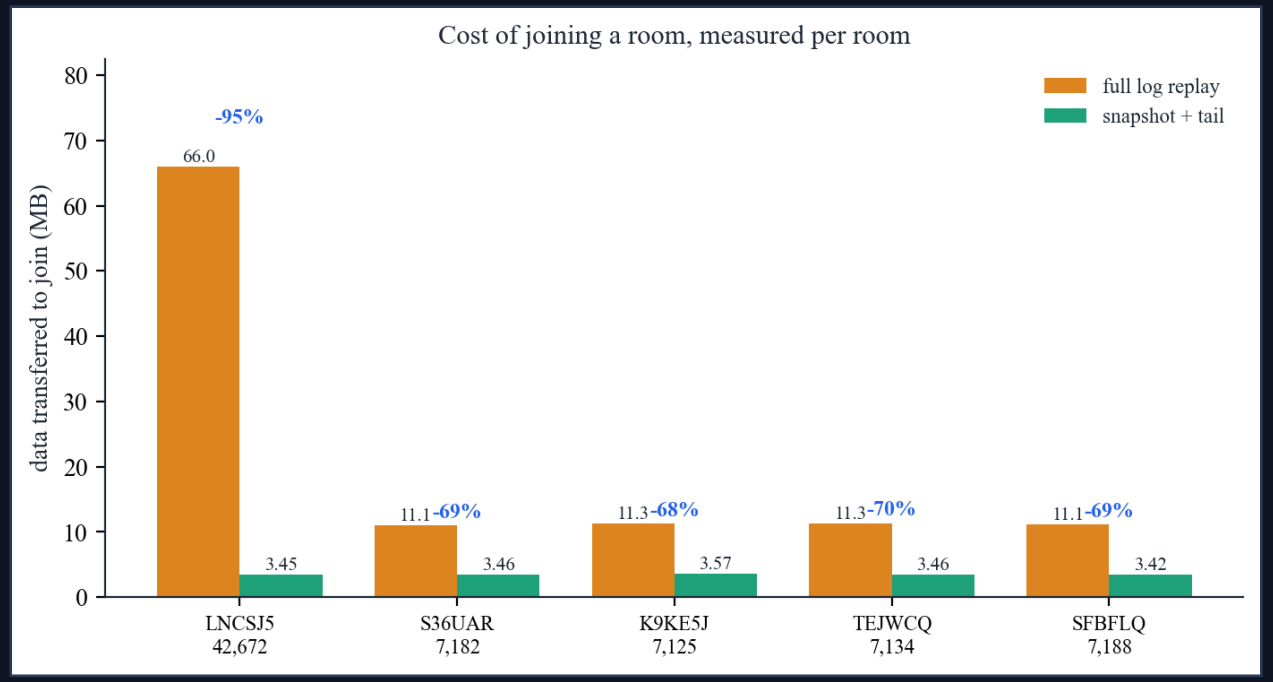
$facet — four summaries, one pass

```
{"$facet": {
   "byTool":    [{"$group": ...}, {"$sort": ...}],
   "byUser":    [{"$group": ...}, {"$limit": 5}],
   "perMinute": [{"$group": {"_id":
                   {"$dateTrunc": ...}}}],
   "bounds":    [{"$group": {"_id": null,
                   "minX": {"$min": "$bbox.minX"}}}]
}}
```

The panel needs four summaries at once. Four queries would mean four round trips and four passes over the same documents; $facet evaluates all four over one $match, so the panel loads from a single request.

QUERY QUALITY

Measurement changed the design



66 MB

to join the 42,672-stroke room by full replay

3.45 MB

with a snapshot plus only the newer strokes

95%

less data transferred; the board opens in about half a second

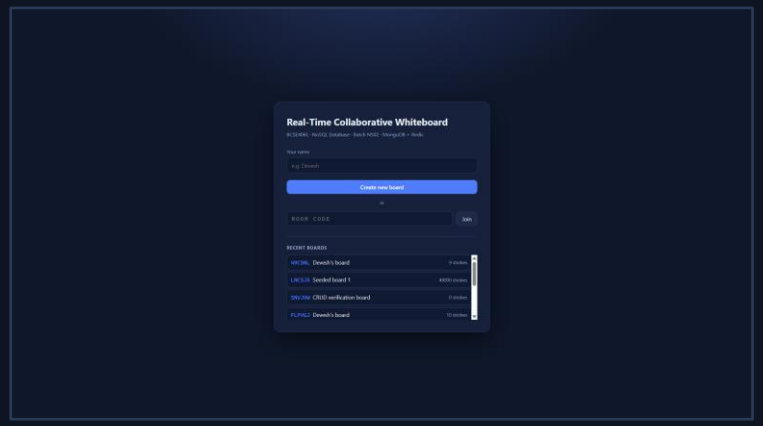
Review 1 called snapshotting optional. The measurement settled it.

Serialising a full replay took seconds, not milliseconds. The snapshot is a cache of derived state — the log still holds the truth, so undo and analytics are unaffected. Below a few hundred strokes the image costs more than the strokes, so small rooms are skipped.

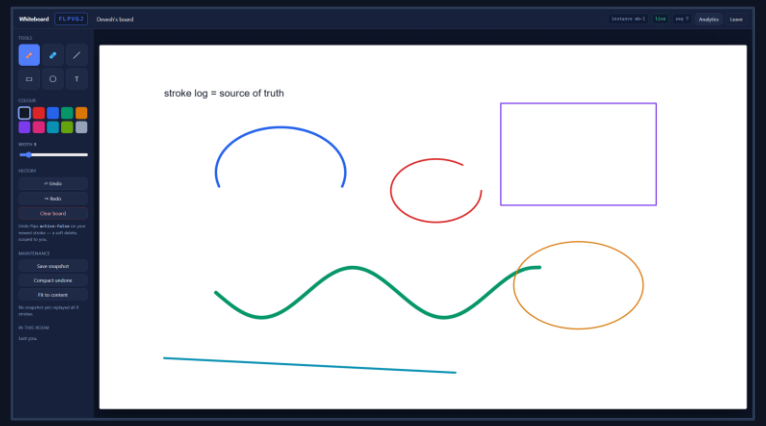
Section 7.2 of the report

PROTOTYPE / DEMO

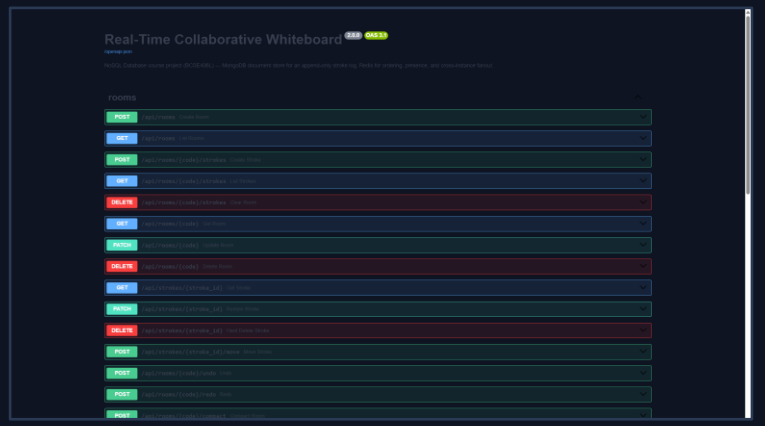
The running prototype



Lobby — boards listed by last activity, served by ix_room_activity



Six tools on one board; every mark is a separate document

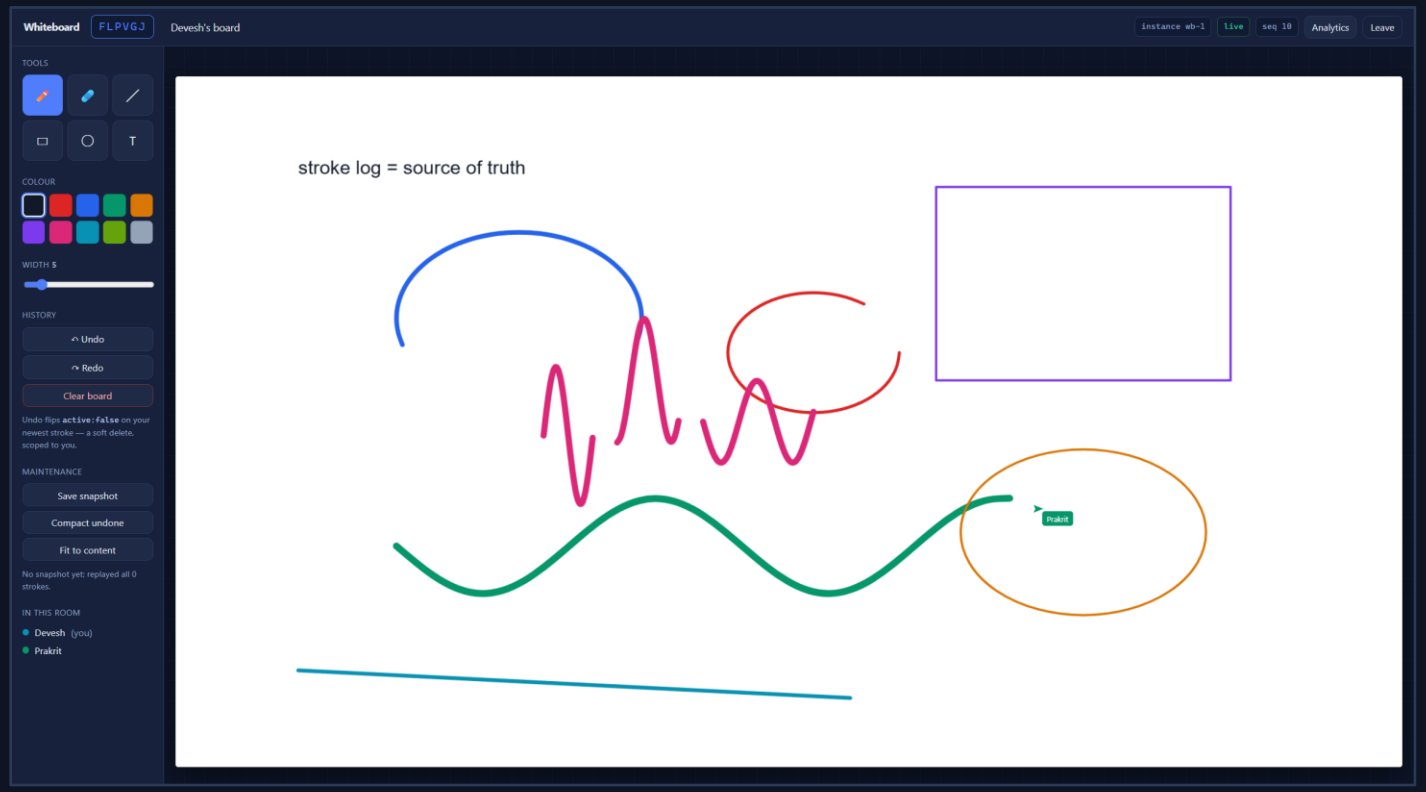


The CRUD surface, exercised in Postman

Section 8 of the report

PROTOTYPE / DEMO

The result that matters most



Two processes, one board

This window is on wb-1, shown in the badge at the top right.

The pink strokes and the labelled cursor belong to a user connected to wb-2 — a separate OS process sharing no memory with this one.

Both arrived through Redis Pub/Sub. This is the claim the architecture makes, and it is the thing a single process could never demonstrate.

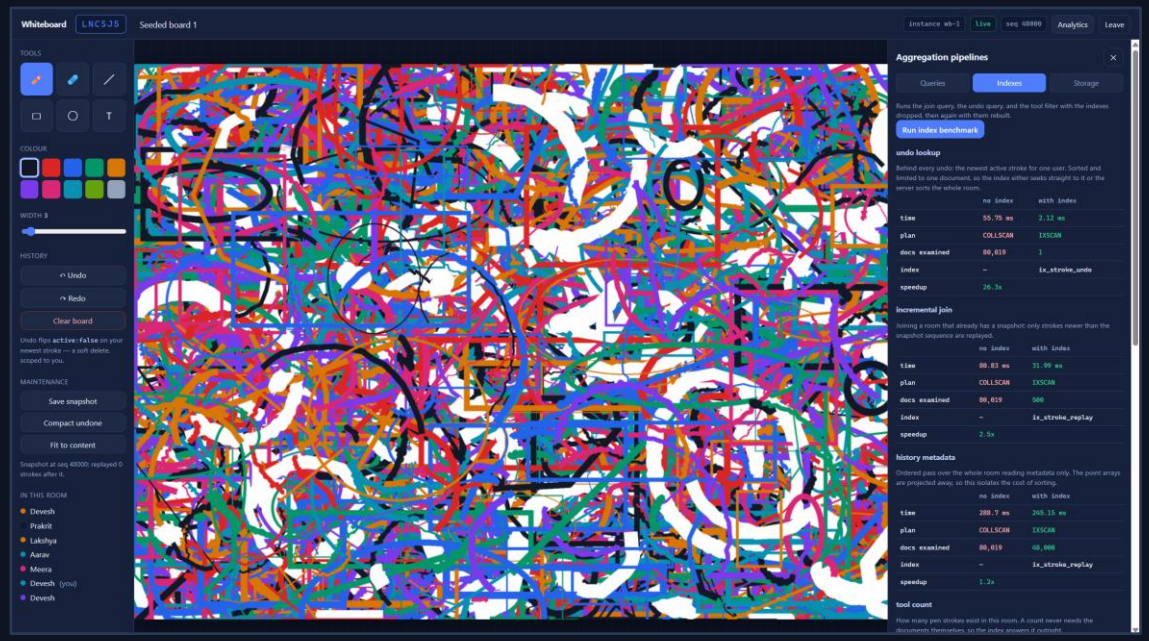
Live in the demo: two browsers, two ports, one room code

PROTOTYPE / DEMO

Database work, visible in the application



The analytics panel shows each pipeline next to its own output and the server-side execution time.



The index benchmark, run from the interface: plan, documents examined and time, with indexes dropped then rebuilt.

Aggregation and indexing are not shell transcripts pasted into a report — they are features of the product, which is why they can be demonstrated live.

Demonstrated live: pick a pipeline, run it, read the plan

DOCUMENTATION & PROGRESS

Progress, and four honest deviations

Completed since Review 1

Application built — backend, transport, canvas client, six tools, presence.

Three collections with several document shapes in one log.

Full CRUD over REST and WebSocket, plus a cascading delete.

Seven indexes with plans captured through explain.

Nine pipelines surfaced in the product, not just a shell.

80,019 documents generated to a realistic shape.

Cross-instance sync demonstrated on two processes.

Deviations from the Review 1 sheet

Review 1 said	Review 2 does
Two collections	Three — join cost forced snapshots
Snapshotting optional	Implemented, client and server side
Atlas free tier	Local Docker; Atlas by configuration
Postman for testing	Postman plus a scripted run, which can express the undo race

Section 9 of the report — 25-page document submitted alongside this deck

DOCUMENTATION & PROGRESS

Limitations we are not hiding, and what comes next

Known limitations

No authentication. Identity is a display name in browser storage, so per-user undo trusts the client.

Preview fanout grows with the square of participants in a room.

Dense snapshots approach the size where GridFS would be right; the headroom is measured, not generous.

No conflict resolution for concurrent edits to one stroke — last write wins. This is exactly where a CRDT would be needed, and it is not claimed.

Timings are cache-warm, which understates the index benefit.

Plan for Review 3

Deploy to Atlas M0 and re-measure; a constrained cache should widen the index advantage.

Add a TTL index to expire abandoned rooms — a NoSQL feature not yet exercised.

Try change streams as a second, database-driven notification path, and compare it with Redis.

Move snapshots to GridFS and compare WebP against PNG.

Add draggable objects, which would make a CRDT layer necessary rather than ornamental.

Load-test the fanout to find where one instance saturates.

Sections 9.3 and 9.4 of the report

LIVE DEMONSTRATION

What we will show in the demo

1. **Create a board, draw with all six tools**
   Each mark becomes one document; watch the stroke count rise.
2. **Open the same board in a second browser on port 8002**
   A different process. Strokes and cursors cross through Redis Pub/Sub.
3. **Undo and redo from both windows**
   Soft delete, scoped per user, claimed atomically.
4. **Exercise CRUD in Postman against the live board**
   A REST insert appears on the canvas without a refresh; cascade delete reports its counts.
5. **Run pipelines from the analytics panel**
   Pipeline and output side by side, with server-side timings.
6. **Run the index benchmark, then open the 42k-stroke board**
   COLLSCAN against IXSCAN, then a snapshot join in about half a second.

Thank you. · Devesh Atul Mahajan (23BCE0801) · Prakrit Bahl (23BCE0826) · Lakshya Diwani (23BCE0811)